

SQL Injection

05 July 2026 11:43

SQL injection is a code injection technique used to executed malicious SQL statements.

How SQL injection works?

username abc@xyz.in password 123456

select * from users where username = "abc@xyz.in" and password = '123456'

user-name password user input.

OR gate logic

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

we use this logic.

'OR 1=1' --' to comment the sql
→ always returns true

How to use sql injection?

GET :- Data is sent in the url of the request.
localhost/index.php? username=admin & password=admin

POST :- Data is sent in the request body of request method.

SQL Injection: SQL injection is a code injection technique where SQL is injected by user (as the part of user input) into the backend query. Inject SQL data alters the purpose of original query and upon execution can gives harmful result.

A SQL injection attack is very dangerous and attacker can,

1. Read sensitive data from the database.
2. Update database data (Insert / update / Delete)
3. Slowdown the complete database system. etc.

username: 'sateh'. delete from user where id=1 user table.

"select * from user where username=''+
username + '";

id	username	password
1	Satyen	mishra
2	Subha	sarkar
3	Baishaki	mishra
4	Nitin	poddar

select * from user where username='satyen';

delete from user where id='1';

At server side, generally, a query is not complete without user data.

Let's understand by taking one example, say you want to get user details for that query will be:

"select * from user where username=''+username+'";

after compilation

select * from user where username='satyen';

↑ attribute in table user.

Is the above query complete and ready to execute?

No, there is dependency on user data.

'username' is the place holder, which is going to be replaced by actual user name of the person stored in our database.

We ask user to enter his/her user name, which is going to fill our placeholder user name.

Enter your username: |

Do we have control on what the user is going to fill in username?

No, say user has entered.

user can fill anything, like name of animal, hospital etc. or can fill some junk data say 'aahh' and press enter or submit button. (username = \$\$\$^aahh*) or user can fill query like say username='satyen'; delete from user where id=1;

Server:-

1. { username = dog. Query will turn into

select * from user where username='dog';

2. { username = apollo. Query will turn into

select * from user where username='apollo';

3. { username = \$\$\$^aahh* Query will turn into

- 3 { username = '\$\$##^ aahh* Query will turn into
 select * from user where username = '\$\$##^ aahh*';
- 4 { username = satyen; Delete from user where id = 1
 Query will turn into
 select * from user where username = 'satyen';
 delete from user where id = '1';

How our application will behave in above cases?

1. For case 1, 2, 3 it may or may not harmful behavior but for case 4, it is sure that we are in trouble.
 Appending user data to our query template, turned query template into two queries and our SQL server engine will execute 2 queries separately.

1. Select Query: select * from user where username = 'satyen';

2. Delete Query: delete from user where id = '1';

This indirect way of injecting SQL code inside the query is called SQL injection.

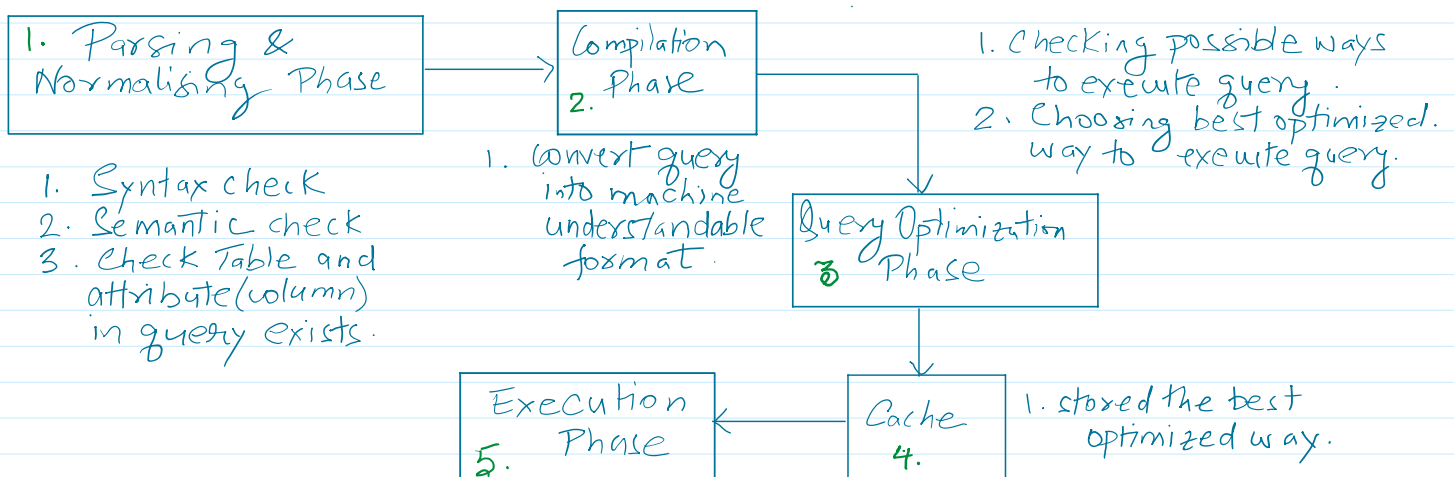
How Prepared Statement in Java Prevents SQL injection?

To under this, Let's see steps involved in query execution.

1. Compilation Phase

2. Execution Phase

Whenever SQL server engine receives a query it has to pass through below phases.



After compile & before execution phase. In this state query is

After compile & before execution phase. In this state query is called Pre-compiled query or already compiled query.

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
public class SQLInjection {
    public static void main(String[] args) {
        String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
        String username = "sys as sysdba";
        String password = "Calcutta1983";
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter username : ");
        String user = sc.nextLine();
        System.out.println("Enter password: ");
        String pass = sc.nextLine();
        try {
            Connection con = DriverManager.getConnection(dbUrl, username, password);
            Statement st = con.createStatement();
            // String sql = "SELECT * FROM USERS WHERE username='"+user+
            // "' And password = '"+pass+"'";
            String sql = "SELECT * FROM USERS WHERE username='"+user+
            "' OR '1' = '1'";
            System.out.println(sql);
            System.out.println(sql);
            ResultSet rs = st.executeQuery(sql);
            if(rs.next()){
                System.out.println("Login Successful");
            }
            else{
                System.out.println("Invalid user name or");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Enter username :
admin
Enter password:
782
SELECT * FROM USERS WHERE username='admin' O
R '1' = '1'
SELECT * FROM USERS WHERE username='admin' O
R '1' = '1'
Login Successful
```

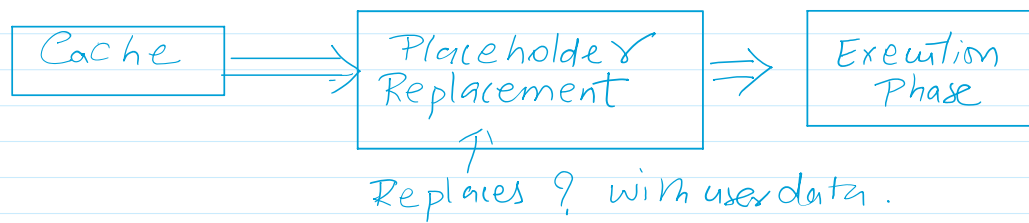

1. Parsing and Normalization Phase: In this phase, Query is checked for syntax and semantics. It checks whether references table and columns used in query exist or not.
2. Compilation Phase: In this phase, Keywords used in the query like select, from, where etc are converted into format understandable by machine. This is the phase where query is interpreted and corresponding action to take is decided.
3. Query Optimization Phase: In this phase, Decision Tree is created for finding the ways in which query can be executed and the cost associated with each way of executing query. It chooses the best plan for executing query.
4. Cache: Best plan selected in query optimization phase and is stored in cache, so that whenever next time same query comes in, it doesn't have to pass through Phase-1, Phase-2 and Phase-3. When next query comes in, it will be checked directly in cache and picked up from there and execute.
5. Execution Phase: In this phase, supplied query gets executed and data is returned to the user as ResultSet object.

Behavior of PreparedStatement API on above steps.

1. PreparedStatement is not complete SQL query and contains placeholders which at runtime is replaced by actual data.
2. Whenever any PreparedStatement containing placeholders is passed into SQL server engine it passes through below phases:
 - (1) Parsing & Normalization. (2) Compilation Phase.
 - (3) Query Optimization Plan (4) Cache (compiled query with placeholder are stored in cache)

" Update user set username = ? and password = ? where id = ? "
3. Above query will get passed, compiled with placeholders as special treatment, optimized and get cached. Query at this stage is already compiled and converted in machine understandable format. So we can say that Query stored in cache is Pre-compiled means already compiled and only placeholders needs to be replaced with user data.

4. Now at run-time when user data comes in, Pre-compiled Query is picked up from cache and placeholders are replaced with user data.



After replacement, Query string formed is not recompiled again. If Query string is not recompiled again, it means, if user data contains SQL then it will not run and will be treated as pure data.

Query stored in cache is Pre-compiled, so it doesn't have to compile again, only replace placeholders and execute query.

(Remember, after placeholders are replaced with user data, final query is not compiled/interpreted again and SQL server engine treats the user data as pure data and not a SQL that needs to be parsed or compiled and that is the beauty of Prepared Statement).

If the query doesn't have to go through compilation phase again, then whatever data is replaced to the placeholders are treated as pure data and has no meaning to SQL server engine and it directly executes the query.

Due to one time compilation feature of Prepared Statement. It is free from SQL Injection Attack.

```
boolean login(String user, String pwd) {
```

```
    boolean loggedIn = false;
```

```
    conn = pool.getConnection();
```

```
    PreparedStatement ps = conn.prepareStatement(  
        "Select * From members where u=? and p=?");
```

```
    // ... rest of the code ...
```

1. `PreparedStatement ps = ...`
"Select * From members where u=? and p=?";

`ps.setString(1, user); ps.setString(2, pwd);`

`ResultSet rs = ps.executeQuery();`

`if (rs.next()) {`
 `loggedIn = true;`
`}`

`return loggedIn;`

`}`